IAI600: Introduction to AI & Machine Learning

# Student Mastery & Operations Handbook

**Group 21 Project:** LLM Analysis and Prediction (Detecting AI-Generated Text)
A complete guide to understanding the codebase, report contents, files, and operations.

⚠️ **Important Course Rules Clarification:**

- **Group Project:** ONLY the written report (maximum 5 pages) must be submitted before November 2. **No oral presentation or demonstration of the project is required.**
- **Individual Oral Demonstration:** Covers **Labs 1, 2, 3, and 5 ONLY** (Labs 4, 6, and 7 are excluded). Each student must demonstrate running code and answer technical questions on Data Preprocessing, Regression, Classification, and Clustering individually.

## Table of Contents


1. **Project Files & Directory Architecture** — Where everything is and what each file does
2. **Code Walkthrough (Step-by-Step for Beginners)** — Line-by-line understanding of every cell
3. **Report Breakdown & Interpretations** — Understanding tables, graphs, and claims
4. **How to Run, Save, and Close Everything** — Step-by-step terminal & browser commands
5. **Understanding the High Accuracy & Baseline Analysis** — Why the model scores 99.6%


## 1. Project Files & Directory Architecture

Your project is self-contained inside `LLM-Analysis-Prediction/` (and mirrored in `~/Downloads/handson-ml3-main/Group21_LLM_Project/`). Here is what every folder and file contains:

| Directory / File | Type | What It Does & Why It Exists |
|---|---|---|
| `data/train_essays.csv` | Raw Data | Original Kaggle competition dataset (1,378 essays: 1,375 human, only 3 AI). |
| `data/daigt_external_dataset.csv` | Raw Data | Paired corpus containing 2,421 student essays and 2,421 matching ChatGPT generations. |
| `data/balanced_essays.csv` | Processed Data | **Unified, cleaned dataset (6,220 essays: 3,796 human, 2,424 AI)** created by merging both sources to fix the class imbalance. |
| `notebooks/llm_analysis_and_prediction.ipynb` | Jupyter Notebook | **Your main interactive notebook.** Contains all markdown explanations, live Python code, printed outputs, and embedded plots. |
| `figures/` | Visualizations | Folder containing 9 publication-grade figures (target distribution, word lengths, confusion matrices, ROC curves, feature weights). |
| `report/IAI600_Group21_Project_Report.pdf` | Submission Document | **Your official 3-page IEEE two-column report** strictly complying with course guidelines (under 5 pages). |
| `report/metrics.json` | Results Data | JSON file storing the exact numerical metrics (Accuracy, F1, ROC-AUC, baseline scores, top features) computed by your models. |
| `run_pipeline.py` | Executable Script | Standalone Python script that runs the entire ML pipeline from the terminal in ~20 seconds using leak-free scikit-learn Pipelines. |
| `generate_report_pdf.py` | Utility Script | Compiles the HTML report and embeds all high-res figures into the final IEEE PDF. |
| `README.md` | Documentation | Repository overview, installation commands, results table, and citations. |
| `LICENSE` | License | Official MIT License for the open-source codebase. |

## 2. Code Walkthrough (Explained for Beginners)

The entire project is organized into clear logical steps:

### Step 1: Imports — Gathering the Tools

```
import pandas as pd             # Used to load, filter, and inspect CSV tables
import numpy as np              # Used for fast mathematical matrix calculations
import matplotlib.pyplot as plt # Used to draw line plots, bar charts, and histograms
import seaborn as sns           # Makes plots look modern with smooth KDE density curves
from sklearn.pipeline import Pipeline                      # Chains vectorizer and model
leak-free
from sklearn.feature_extraction.text import TfidfVectorizer # Converts text words into
numbers
from sklearn.linear_model import LogisticRegression       # Classification algorithm
from sklearn.naive_bayes import MultinomialNB              # Generative probability
algorithm
from sklearn.ensemble import RandomForestClassifier       # Ensemble decision tree
algorithm
from sklearn.svm import LinearSVC                          # Maximum margin
classification algorithm
```

### Step 2: Loading & Merging Datasets

We read both CSV files. In `train_essays.csv`, student essays have `generated = 0`. In `daigt_external_dataset.csv`, student essays are labeled `0` and ChatGPT responses (source text) are labeled `1`. We combine them using `pd.concat`, retaining all 6,220 essays.

### Step 3: Exploratory Data Analysis (EDA)

- **Word count per essay:** Students wrote longer essays (~481 words) than the prompt-constrained ChatGPT essays (~204 words).
- **True average word length (excluding spaces):** AI essays used longer, more polysyllabic words (5.01 letters/word vs. 4.32 letters/word for students).
- **Sentence count:** Student essays contained more sentences (~22 vs. ~10).

### Step 4: Text Cleaning (Preprocessing)

```
def preprocess_text(text):
    text = text.lower()                                  # 1. Lowercase all letters
    text = re.sub(r'http\S+|www\S+', '', text)           # 2. Remove web links
    text = re.sub(r'[^a-z0-9\s]', ' ', text)             # 3. Strip punctuation
    text = re.sub(r'\s+', ' ', text).strip()            # 4. Normalize spaces
    return text
```

### Step 5: TF-IDF Feature Extraction

- **TF (Term Frequency):** How often a term appears in an essay.
- **IDF (Inverse Document Frequency):** How rare the term is across the entire corpus of 6,220 essays.
- **Stopword removal:** Removes common filler words ("*the*", "*is*", "*however*").
- **N-gram range (1, 2):** Extracts single words and adjacent two-word combinations.

### Step 6: Train/Test Split & Pipeline Cross-Validation

- **80% Training (4,976 essays) / 20% Testing (1,244 essays).**
- `Pipeline` ensures that the TF-IDF vectorizer is fitted strictly on training data during each of the 5 cross-validation folds, preventing any data leakage.

### Step 7: The 4 Machine Learning Models & Results

| Model Architecture | 5-Fold CV Accuracy | Test Accuracy | Precision | Recall | F1-Score | ROC-AUC |
|---|---|---|---|---|---|---|
| **Linear SVM (Calibrated)** | 99.50% ± 0.14% | **99.60%** | **99.38%** | **99.59%** | **99.49%** | **0.9999** |
| **Logistic Regression** | 99.38% ± 0.26% | 99.36% | 99.18% | 99.18% | 99.18% | 0.9996 |
| **Random Forest (100 trees)** | 99.30% ± 0.28% | 99.36% | 99.18% | 99.18% | 99.18% | 0.9988 |
| **Multinomial Naive Bayes** | 98.55% ± 0.53% | 98.07% | 96.19% | 98.97% | 97.56% | 0.9989 |

### Step 8: Verified Feature Importance (Top Words)

By inspecting the Logistic Regression model coefficients, we observe the exact tokens driving classification (matching Figure 6 in the report):

- **Top AI-predictive words:** `potential` (+2.88), `provide` (+2.49), `ultimately` (+2.38), `additionally` (+2.23), `essential` (+2.14), `difficult` (+1.95), `understanding` (+1.88).
- **Top Human-predictive words:** `want` (-4.07), `good` (-3.91), `like` (-3.70), `people` (-3.68), `think` (-3.66), `know` (-2.93), `don` (-2.75), `thing` (-2.62), `just` (-2.54).

---

## 3. Understanding the High Accuracy & Baseline Analysis

> 💡 **Why is the accuracy so high (99.6%)?**
>
> A smart teacher will ask: *"99.6% accuracy sounds almost too good. Did your model learn genuine authorship, or did it just learn essay length?"*
>
> **Your Explanation (backed by our baseline experiment):**
> We ran a controlled baseline experiment using **ONLY 5 structural features** (word count, letter count, word length, sentence count, and average sentence length) without any text words. That length-only model achieved **95.42% accuracy** and an ROC-AUC of 0.9908!
>
> This proves that macro-structural differences (brevity vs. elaboration) account for ~95% of the class separation. However, lexical TF-IDF features provide the crucial extra boost from 95.42% to 99.60%, eliminating false positives on essays of unusual length.

---

## 4. How to Run, Save, and Close Everything

### A. How to Open the Jupyter Notebook

```
jupyter notebook notebooks/llm_analysis_and_prediction.ipynb
```

### B. How to Run Cells

- Click on any code cell and press **Shift + Enter**.
- To re-run everything: click **Kernel → Restart & Run All Cells**.

### C. How to Save & Stop

- Save: press **Cmd + S**.
- Stop: close browser tab, switch to Terminal, and press **Ctrl + C**, then type `y`.

### D. How to Re-Run the Entire Pipeline from Terminal

```
python3 run_pipeline.py
```

### E. How to Re-Compile the IEEE Report PDF

```
python3 generate_report_pdf.py
```

---

## 5. Checklist Before Submitting on Canvas

- [ ] Open `report/IAI600_Group21_Project_Report.pdf` and check that all group members' names and IDs are entered in the title block. (Edit `generate_report_pdf.py` and run it to update).
- [ ] Push your project folder to your GitHub repository and ensure the link matches Reference [8].
- [ ] Submit the PDF report on Canvas before the November 2 deadline.
- [ ] Study the separate **Oral Demonstration Guide for Labs 1, 2, 3, and 5** to prepare for your individual oral test.